

Praveen Madupu

---

# AWS Certified Solutions Architect Professional (SAP-C02)

*Interview Preparation Short notes*

7 Chapters • 47 Topics • Quick-Reference Notes  
Org Complexity | New Solutions | Continuous Improvement | Migration

**Drafted by: Praveen Madupu**

## Table of Contents

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| Table of Contents .....                                                          | 2  |
| Chapter 1 — Organizational Complexity: Multi-Account Governance & Security ..... | 4  |
| 1.1 AWS Organizations & OU Design .....                                          | 4  |
| 1.2 Service Control Policies (SCPs) & Guardrails .....                           | 4  |
| 1.3 Control Tower & Landing Zones .....                                          | 4  |
| 1.4 Cross-Account IAM, Roles & Federation .....                                  | 5  |
| 1.5 Advanced IAM: ABAC, Boundaries & Policy Evaluation .....                     | 5  |
| 1.6 KMS at Scale & Multi-Region Keys .....                                       | 5  |
| 1.7 Centralized Security & Governance .....                                      | 6  |
| Chapter 2 — Advanced Networking & Hybrid Connectivity .....                      | 7  |
| 2.1 VPC Design & IP Address Planning .....                                       | 7  |
| 2.2 Transit Gateway & Network Segmentation .....                                 | 7  |
| 2.3 Direct Connect Deep Dive .....                                               | 7  |
| 2.4 Site-to-Site VPN & Hybrid Patterns .....                                     | 8  |
| 2.5 Hybrid & Multi-Account DNS (Route 53 Resolver) .....                         | 8  |
| 2.6 PrivateLink, Endpoints & Cross-VPC Access .....                              | 8  |
| 2.7 Global Traffic Management & Edge .....                                       | 8  |
| Chapter 3 — Advanced Compute, Containers & Serverless .....                      | 9  |
| 3.1 EC2 Advanced (Nitro, Fleets, Capacity) .....                                 | 9  |
| 3.2 Auto Scaling & Compute Optimization at Scale .....                           | 9  |
| 3.3 Serverless Architectures (Lambda, Step Functions, EventBridge) .....         | 9  |
| 3.4 API Gateway & AppSync Patterns .....                                         | 9  |
| 3.5 ECS / EKS at Scale & Fargate .....                                           | 10 |
| 3.6 Event-Driven & Decoupled Architectures .....                                 | 10 |
| 3.7 Caching & Performance .....                                                  | 10 |
| Chapter 4 — Data, Storage & Database Architectures .....                         | 11 |
| 4.1 Amazon S3 Advanced .....                                                     | 11 |
| 4.2 Block & File Storage at Scale .....                                          | 11 |
| 4.3 RDS & Aurora Advanced .....                                                  | 11 |
| 4.4 DynamoDB Advanced .....                                                      | 11 |
| 4.5 Analytics & Big Data .....                                                   | 12 |
| 4.6 Redshift & Data Warehousing at Scale .....                                   | 12 |
| 4.7 Purpose-Built Databases & Migration Tools .....                              | 12 |
| Chapter 5 — Resilience, Disaster Recovery & High Availability .....              | 13 |
| 5.1 Multi-AZ & Multi-Region Architectures .....                                  | 13 |
| 5.2 Disaster Recovery Strategies & RTO/RPO .....                                 | 13 |
| 5.3 Backup, Replication & Data Durability .....                                  | 13 |
| 5.4 Fault Isolation & Resilience Patterns .....                                  | 13 |

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 5.5 Observability at Scale .....                                       | 14 |
| 5.6 Chaos Engineering & Fault Injection .....                          | 14 |
| Chapter 6 — Migration & Modernization .....                            | 15 |
| 6.1 Migration Strategy & the 7 Rs .....                                | 15 |
| 6.2 Discovery & Planning .....                                         | 15 |
| 6.3 Server & Database Migration .....                                  | 15 |
| 6.4 Data Transfer (DataSync, Transfer Family, Snow) .....              | 15 |
| 6.5 Modernization Patterns .....                                       | 16 |
| 6.6 Hybrid & VMware Cloud .....                                        | 16 |
| Chapter 7 — Cost Optimization, Deployment Automation & Exam Prep ..... | 17 |
| 7.1 Cost Optimization at Scale .....                                   | 17 |
| 7.2 Cost Governance & Visibility .....                                 | 17 |
| 7.3 CI/CD & Deployment Automation .....                                | 17 |
| 7.4 Deployment Strategies .....                                        | 17 |
| 7.5 Infrastructure as Code .....                                       | 18 |
| 7.6 SAP-C02 Exam Overview & Domains .....                              | 18 |
| 7.7 Exam Strategy & Career Path .....                                  | 18 |

## Chapter 1 — Organizational Complexity: Multi-Account Governance & Security

### 1.1 AWS Organizations & OU Design

- Org = management (payer) account + members grouped into OUs (nested up to 5 levels). Enables consolidated billing, shared Savings Plans/RIs, and org-wide policy.
- Recommended OU structure (AWS): Security OU (Log Archive + Audit accounts), Infrastructure OU (shared networking/services), Workloads OU (prod/non-prod), Sandbox, Suspended, and a Policy Staging OU.
- Isolate blast radius per account: separate prod/dev, per-team accounts, dedicated security tooling account. Accounts are the strongest isolation boundary (quotas, billing, IAM).
- Delegated administrator: offload service admin (GuardDuty, Config, Security Hub, IPAM) from the management account to a member account — keep the management account minimal.

**TIP** Never run workloads in the management account. SCPs don't apply to it, and it holds org-wide billing/root power.

- Policy types beyond SCPs: Tag policies (enforce tag standards), Backup policies (org-wide AWS Backup), AI services opt-out policies.

### 1.2 Service Control Policies (SCPs) & Guardrails

- SCPs are permission guardrails (maximum boundary) — they never grant, only limit. Effective permission = intersection of SCP AND IAM identity policy.
- Two strategies: allow-list (deny all, allow approved — tight but high maintenance) or deny-list (allow all, deny specific — default, easier). Deny-list is most common.

```
// Deny disabling of security controls (region + service guardrail)
{ "Effect": "Deny",
  "Action": [ "cloudtrail:StopLogging", "config:DeleteConfigurationRecorder",
             "guardduty:DeleteDetector" ],
  "Resource": "*" }
```

- Common guardrails: restrict regions (aws:RequestedRegion condition), block root user actions, deny leaving the org, enforce EBS/S3 encryption, require IMDSv2, restrict instance types.

**WATCH** SCPs do NOT affect service-linked roles and don't restrict the management account. An explicit Deny in an SCP overrides any IAM Allow.

- Permission boundary (IAM, per-identity) vs SCP (Organizations, per-account/OU) — layer both for delegated admin safety.

### 1.3 Control Tower & Landing Zones

- Control Tower automates a secure multi-account landing zone: Account Factory (provisioning), preventive guardrails (SCPs) + detective guardrails (Config rules), centralized logging (Log Archive) and Audit accounts.
- Guardrail categories: Mandatory (always on), Strongly Recommended, Elective. Proactive guardrails (CloudFormation Hooks) block non-compliant resources before creation.
- Use Control Tower for a governed baseline; use Customizations for Control Tower (CfCT) or Landing Zone Accelerator (LZA) for large/regulated customizations.

**TIP** Account Factory for Terraform (AFT) provisions accounts via GitOps pipelines — good when IaC/Terraform is the standard.

- AWS RAM (Resource Access Manager) shares resources across accounts (subnets via shared VPC, Transit Gateway, License Manager, Route 53 Resolver rules) without duplication.

## 1.4 Cross-Account IAM, Roles & Federation

- Cross-account access = role in target account trusts source account/principal; caller does sts:AssumeRole. Add ExternalId for third parties to prevent the confused-deputy problem.
- IAM Identity Center (successor to AWS SSO) = central workforce identity: connect to Okta/Azure AD/AD, define permission sets, assign to accounts/OUs. Users get short-lived creds.
- Federation: SAML 2.0 (enterprise IdP → AssumeRoleWithSAML), OIDC/web identity (Cognito → AssumeRoleWithWebIdentity), and IAM Identity Center for workforce.
- SCPs + permission boundaries + session policies + resource policies all narrow effective permissions — reason about the intersection.

**TIP** For app-to-AWS, prefer roles (EC2 instance profile, EKS IRSA, Lambda execution role) over long-lived keys. IRSA maps K8s service accounts to IAM roles via OIDC.

## 1.5 Advanced IAM: ABAC, Boundaries & Policy Evaluation

- ABAC (attribute-based access control) uses tags in conditions (aws:PrincipalTag, aws:ResourceTag, aws:RequestTag) to scale permissions without new policies per team/project.

```
// ABAC: allow only when principal tag matches resource tag
"Condition": { "StringEquals": {
  "aws:ResourceTag/team": "${aws:PrincipalTag/team}" } }
```

- Policy evaluation order: explicit Deny > Org SCP > resource policy > permission boundary > session policy > identity policy. Any Deny anywhere wins; a request needs an Allow with no blocking Deny.
- Permission boundaries cap what an identity can be granted (delegated admins can create roles but only within the boundary).
- IAM Access Analyzer: finds resources shared externally, validates policies, and generates least-privilege policies from CloudTrail activity.

**WATCH** Resource-based policies (S3, KMS, SNS, SQS, Lambda) can grant cross-account access directly — the caller doesn't always need an identity-based Allow for cross-account (except IAM roles/users).

## 1.6 KMS at Scale & Multi-Region Keys

- KMS multi-region keys share the same key material + key ID across regions — enables encrypting in one region and decrypting in another (DynamoDB Global Tables, cross-region DR, S3 CRR).
- Key policy is authoritative for KMS access; combine with grants (temporary, programmatic) and IAM. Cross-account KMS needs key policy + IAM in the caller account.
- Reduce cost/throttling: S3 Bucket Keys (fewer KMS calls), data-key caching, and choosing the right key granularity. KMS has per-region request quotas.
- CloudHSM = single-tenant FIPS 140-2 Level 3 when you must own/operate the HSM (custom key store integrates CloudHSM with KMS).

**TIP** Multi-Region keys are for cross-region crypto interoperability — they are NOT global; each replica is managed independently with its own policy.

## 1.7 Centralized Security & Governance

- Delegate and centralize: GuardDuty, Security Hub, Inspector, Macie, Config, and IAM Access Analyzer all support org-wide + delegated admin aggregation into a security account.
- AWS Config aggregators collect compliance across accounts/regions; conformance packs deploy rule sets org-wide. Config + EventBridge + SSM Automation = auto-remediation.
- Centralized logging: org CloudTrail (management + data events) → central S3 in Log Archive; VPC Flow Logs + DNS logs to a central account.
- Firewall Manager centrally manages WAF rules, Shield Advanced, security groups, and Network Firewall policies across the org.

**TIP** Pattern: Log Archive account (immutable logs, Object Lock) + Audit/Security account (read-only cross-account roles, aggregated findings).

## Chapter 2 — Advanced Networking & Hybrid Connectivity

### 2.1 VPC Design & IP Address Planning

- Plan non-overlapping CIDRs across VPCs/accounts/on-prem (peering + TGW require no overlap). Use AWS IPAM to allocate, track, and audit address space org-wide.
- Subnet tiers: public (IGW route), private (NAT for egress), isolated (no internet — DBs). Reserve 5 IPs/subnet. Add secondary CIDRs when you run out.
- NAT Gateway is AZ-scoped — deploy one per AZ and route each AZ's private subnets to its local NAT for HA + to avoid cross-AZ data charges.
- Egress cost control: VPC endpoints (S3/DynamoDB gateway = free) and interface endpoints avoid NAT/internet egress; centralized egress VPC via TGW consolidates NAT.

**TIP** Shared VPC (RAM): a central networking account owns the VPC/subnets and shares them to workload accounts — one place to manage routing/security.

### 2.2 Transit Gateway & Network Segmentation

- Transit Gateway (TGW) = regional hub for many VPCs + VPN + Direct Connect, with transitive routing — replaces  $N^2$  VPC peering mesh at scale.
- Segmentation via multiple TGW route tables + associations/propagations: isolate prod/dev, create shared-services domains, control which attachments can reach each other.
- TGW peering connects TGWs across regions (inter-region, encrypted over AWS backbone) for global networks. Appliance mode keeps flow symmetry for inspection.
- Centralized inspection/egress: route spoke VPC traffic through a security VPC (Network Firewall/GWLB appliances) attached to the TGW.

**WATCH** TGW is non-transitive across peering by default for route propagation nuances — plan route tables carefully. Data processing + attachment charges apply per GB/hour.

### 2.3 Direct Connect Deep Dive

- DX = dedicated private physical link (1/10/100 Gbps dedicated, or hosted sub-1G). Consistent latency, higher throughput, lower egress cost than internet.
- Virtual interfaces: Private VIF (to a VPC/VGW), Public VIF (to public AWS services/endpoints), Transit VIF (to a Direct Connect Gateway → TGW).
- Direct Connect Gateway (DXGW) = global, connects a DX to VPCs/TGWs in any region (same account or via association) — one DX serves multiple regions.
- Resiliency tiers: Development (single), High (2 devices/1 location), Maximum (2 devices in 2 locations). Always back DX with a Site-to-Site VPN failover.

**TIP** DX has no encryption by default — layer IPsec VPN over DX (private IP VPN / MACsec on supported ports) for confidentiality + compliance.

## 2.4 Site-to-Site VPN & Hybrid Patterns

- Site-to-Site VPN = IPsec over internet, two tunnels for redundancy, terminates on a VGW (per-VPC) or TGW (scales to many VPCs). Quick to deploy, variable performance.
- Route options: static or BGP dynamic (preferred — auto failover, ECMP for aggregate throughput on TGW). Accelerated VPN uses Global Accelerator edge for better performance.
- Common hybrid patterns: VPN as DX backup, VPN for burst/temporary, DX+VPN for encrypted private connectivity, TGW as the single hybrid attachment point.

**TIP** For >1.25 Gbps per VPN or lowest latency, use DX; use multiple VPN tunnels with BGP ECMP on TGW to scale VPN throughput.

## 2.5 Hybrid & Multi-Account DNS (Route 53 Resolver)

- Route 53 Resolver endpoints bridge DNS between VPC and on-prem: Inbound endpoint (on-prem → resolves AWS private zones), Outbound endpoint + Resolver rules (AWS → forwards to on-prem DNS).
- Share Resolver rules org-wide via RAM so all accounts forward corporate domains to on-prem consistently.
- Private Hosted Zones can be associated with multiple VPCs/accounts; use a central DNS account pattern for shared name resolution.
- Route 53 DNS Firewall blocks/allow-lists domains (exfiltration protection) at the VPC resolver level.

**TIP** Split-horizon DNS: same name resolves to internal IPs inside the VPC (private zone) and public IPs externally (public zone).

## 2.6 PrivateLink, Endpoints & Cross-VPC Access

- PrivateLink exposes a service (behind an NLB or GWLB) privately to consumer VPCs/accounts via interface endpoints — no peering, no route exposure, no overlapping-CIDR concerns.
- Endpoint types: Gateway (S3, DynamoDB — free, route-table entry) vs Interface (ENI + PrivateLink, hourly + per-GB, most services + your own services).
- PrivateLink vs peering/TGW: PrivateLink = one-way access to a specific service (great for SaaS/shared services, overlapping CIDRs OK); peering/TGW = full network routing.

**TIP** Expose an internal microservice to many consumer accounts securely → PrivateLink endpoint service (NLB) rather than opening the whole VPC.

## 2.7 Global Traffic Management & Edge

- Route 53 routing policies for global apps: Latency (nearest region), Geolocation/Geoproximity (compliance/localization + bias), Failover (active-passive), Weighted (canary/blue-green), Multivalue.
- CloudFront = caching CDN + edge compute (CloudFront Functions for lightweight, Lambda@Edge for heavier), OAC to lock S3/ALB origins, origin failover for HA.
- Global Accelerator = anycast static IPs + AWS backbone for non-HTTP/dynamic/TCP-UDP traffic; fast regional failover (health-checked endpoint groups + traffic dials).

**TIP** CloudFront for cacheable HTTP; Global Accelerator for dynamic/non-HTTP, static entry IPs, and sub-minute cross-region failover. They can be combined.

- Multi-region active-active: Route 53 latency/geo + regional stacks + global data layer (Aurora Global DB / DynamoDB Global Tables).



## Chapter 3 — Advanced Compute, Containers & Serverless

### 3.1 EC2 Advanced (Nitro, Fleets, Capacity)

- Nitro system offloads virtualization to dedicated hardware (better perf, EBS/network throughput, Nitro Enclaves for isolated sensitive compute). Graviton (ARM) = ~40% better price/performance.
- Placement groups: Cluster (low latency/high throughput HPC), Spread (max 7/AZ, hardware isolation for critical), Partition (up to 7 partitions/AZ for large distributed like Cassandra/Hadoop).
- EC2 Fleet / Spot Fleet: mix On-Demand + Spot across instance types/AZs; allocation strategies capacity-optimized (fewest interruptions) or lowest-price. Attribute-based instance selection.
- Capacity Reservations (On-Demand, AZ-scoped) guarantee capacity; combine with Savings Plans for the discount. Dedicated Hosts for BYOL/socket-based licensing + compliance.

**TIP** Spot for stateless/fault-tolerant with capacity-optimized allocation + interruption handling; On-Demand Capacity Reservations for guaranteed launch during events/DR.

### 3.2 Auto Scaling & Compute Optimization at Scale

- ASG mixed instances policy blends On-Demand baseline + Spot across many types for cost + resilience. Warm pools cut scale-out latency for slow-booting apps.
- Scaling: target tracking (baseline), predictive (ML forecast for known cycles), scheduled (business hours), step (magnitude-based). Combine predictive + dynamic for spiky/predictable load.
- Right-sizing tooling: Compute Optimizer (ML recommendations across EC2/ASG/Lambda/EBS), Trusted Advisor, and CloudWatch. Graviton migration for cost/perf.

**TIP** Instance refresh + launch template versions enable rolling AMI/config updates across an ASG with health checks and rollback.

### 3.3 Serverless Architectures (Lambda, Step Functions, EventBridge)

- Lambda: 15-min max, 128 MB–10 GB memory (CPU scales with memory), 10 GB /tmp, up to 10 GB container images. Reserved vs provisioned concurrency (pre-warm to kill cold starts).
- Step Functions orchestrate workflows: Standard (long-running, exactly-once, up to 1 yr) vs Express (high-volume, short, at-least-once). Handles retries, parallel, error handling, saga patterns.
- EventBridge = event bus + schema registry + rules + Pipes (point-to-point with filtering/enrichment) + Scheduler. Decouples producers/consumers; SaaS + custom + AWS events.
- Async Lambda: automatic retries (2) + DLQ/on-failure destinations. Event source mapping for SQS/Kinesis/DynamoDB Streams with batching + parallelization factor.

**WATCH** Lambda concurrency default 1000/region (soft). Downstream (RDS connections) can be overwhelmed — use RDS Proxy + reserved concurrency to protect it.

### 3.4 API Gateway & AppSync Patterns

- API Gateway: REST (full features — caching, request validation, WAF, private APIs, usage plans) vs HTTP (lower cost/latency, JWT auth) vs WebSocket (real-time). Private APIs via interface endpoint.

- Auth: IAM/SigV4, Cognito user pools, Lambda authorizers (token/request), API keys + usage plans for throttling/quotas per consumer.
- AppSync = managed GraphQL: real-time subscriptions, offline sync, resolvers to DynamoDB/Lambda/RDS/HTTP; good for mobile/real-time data-fetch consolidation.

**TIP** Use API Gateway caching + throttling to protect backends; mTLS + resource policies + WAF for hardened public APIs. AppSync when clients need flexible GraphQL queries + real-time.

### 3.5 ECS / EKS at Scale & Fargate

- ECS (AWS-native) vs EKS (Kubernetes portability/ecosystem). Fargate (serverless, no nodes) vs EC2 launch type (cheaper at steady high utilization, GPU/custom AMIs, daemonsets).
- EKS networking: VPC CNI gives pods VPC IPs; IRSA maps service accounts → IAM roles; Karpenter/Cluster Autoscaler for node scaling; Fargate profiles for serverless pods.
- Task/pod IAM roles scope permissions per workload. ALB (ingress) + target groups for HTTP; NLB for TCP. ECR with image scanning + lifecycle policies.

**TIP** Fargate for spiky/low-ops and strong isolation; EC2 for cost at scale, GPUs, and node-level control. EKS for multi-cloud/k8s skills; ECS for simpler AWS-integrated ops.

### 3.6 Event-Driven & Decoupled Architectures

- SQS decouples + buffers (Standard = at-least-once/best-effort order/high throughput; FIFO = ordered/exactly-once, 300 or 3000 msg/s batched). Visibility timeout + DLQ for poison messages.
- SNS fan-out (push to SQS/Lambda/HTTP/email/SMS); SNS FIFO for ordered fan-out; message filtering routes by attributes. SNS+SQS = durable multi-consumer fan-out.
- Kinesis Data Streams = ordered, replayable, sharded real-time streaming (multiple consumers, enhanced fan-out); Firehose = managed delivery to S3/Redshift/OpenSearch with transform.
- EventBridge for routing/filtering across many sources + Pipes for source→target with enrichment. Choose by need: buffer (SQS), fan-out (SNS), stream/replay (Kinesis), route (EventBridge).

**TIP** Idempotency + DLQs + retries + backpressure make async resilient. Order requirement → FIFO/Kinesis; massive fan-out → SNS; replay/analytics → Kinesis.

### 3.7 Caching & Performance

- Layer caches: CloudFront (edge/static), API Gateway cache, ElastiCache (DB query/session), DAX (DynamoDB microsecond reads). Cache before scaling out the backend.
- ElastiCache Redis (HA, replication, persistence, cluster mode, pub/sub, sorted sets) vs Memcached (simple, multithreaded, no persistence). Redis for HA/session/leaderboards.
- Strategies: lazy-loading (cache on miss) vs write-through (write to cache+DB); TTLs to bound staleness; cache-aside is the common default.

**TIP** Session state externalized to ElastiCache/DynamoDB makes the app tier stateless → enables clean horizontal scaling and multi-AZ resilience.

## Chapter 4 — Data, Storage & Database Architectures

### 4.1 Amazon S3 Advanced

- Classes by access + cost: Standard, Intelligent-Tiering (auto, no retrieval fee — default when unknown), Standard-IA, One Zone-IA, Glacier Instant/Flexible/Deep Archive. Lifecycle transitions + expiration.
- Replication: CRR (DR/latency/compliance) + SRR (log aggregation/prod-to-test); needs versioning; Replication Time Control (RTC) gives a 15-min SLA. Batch Replication for existing objects.
- Security: Block Public Access (default on), bucket policies, Access Points (+ Multi-Region Access Points for global routing/failover), Object Lock (WORM), SSE-KMS with Bucket Keys.
- Performance: 3,500 write / 5,500 read requests per prefix/sec — parallelize across prefixes; multipart + Transfer Acceleration for large/long-distance; S3 Select + Glacier retrieval tiers.

**TIP** Unknown/variable access → Intelligent-Tiering. Compliance immutability → Object Lock (governance/compliance mode) in a locked Log Archive bucket.

### 4.2 Block & File Storage at Scale

- EBS: gp3 (default, decoupled IOPS/throughput), io2 Block Express (up to 256K IOPS, sub-ms, 99.999%), st1/sc1 HDD (throughput/cold). Multi-Attach (io1/io2) for clustered apps in one AZ.
- EFS = elastic multi-AZ NFS (Linux), Standard + IA lifecycle, Elastic/Provisioned throughput, access points; scales to PB. EBS is single-AZ block; EFS is shared multi-AZ file.
- FSx: Windows (SMB/AD/DFS), Lustre (HPC/ML, S3-linked, scratch/persistent), NetApp ONTAP (multi-protocol, SnapMirror, dedup), OpenZFS (low-latency NFS).

**TIP** Linux shared → EFS; Windows/AD shares → FSx Windows; HPC/ML high throughput + S3 → FSx Lustre; enterprise NAS features/migration → ONTAP.

### 4.3 RDS & Aurora Advanced

- Aurora storage: 6 copies/3 AZs, auto-heal, up to 128 TB, 15 low-lag readers on shared storage. Writer/reader/custom endpoints. Failover ~30s (faster with reader promotion).
- Aurora Global Database: cross-region replication <1s, RPO seconds / RTO <1 min, up to 5 secondary regions; write forwarding for global writes. Backtrack rewinds without restore.
- Aurora Serverless v2: fine-grained ACU autoscaling for variable/spiky workloads (mix with provisioned in the same cluster).
- RDS Proxy pools + multiplexes connections (essential for Lambda/serverless), improves failover, and manages Secrets Manager creds. Blue/Green deployments for low-risk upgrades.

**TIP** Read scaling → replicas; write scaling → shard, offload with caching/queues, or Aurora Limitless (sharded). Global DR → Aurora Global Database.

### 4.4 DynamoDB Advanced

- Design for access patterns first (single-table design). Partition key must be high-cardinality to avoid hot partitions; use composite keys + GSI overloading for multiple query patterns.
- GSI (any key, own capacity, eventually consistent) vs LSI (same PK, alt sort key, created at table creation, strong consistency, shares capacity).

- Global Tables = multi-region, multi-active replication (last-writer-wins) for global apps + regional DR. Streams → Lambda for CDC/event-driven pipelines. TTL auto-expires items.
- Capacity: On-Demand (spiky/unknown) vs Provisioned + auto-scaling (steady, cheaper). DAX for microsecond reads. PITR (35 days) + on-demand backups. Transactions for ACID.

**WATCH** Avoid scans on large tables; use Query with well-designed keys/GSIs. Watch item-collection size limits with LSIs (10 GB per partition key).

#### 4.5 Analytics & Big Data

- Ingest: Kinesis Data Streams (real-time, replay, sharded) / Firehose (managed delivery + transform to S3/Redshift/OpenSearch) / MSK (managed Kafka).
- Process: Glue (serverless ETL + Data Catalog + crawlers), EMR (managed Spark/Hadoop/Hive, Spot-friendly), Lambda for light transforms.
- Query: Athena (serverless SQL on S3, pay-per-scan — partition + columnar Parquet to cut cost), Redshift Spectrum (S3 from Redshift), OpenSearch (log/search analytics).
- Govern: Lake Formation (fine-grained lake permissions + centralized catalog + cross-account sharing). QuickSight for BI dashboards.

**TIP** Data lake pattern: S3 (storage) + Glue Catalog (metadata) + Athena/EMR/Redshift Spectrum (query) + Lake Formation (governance). Columnar + partitioning = big Athena savings.

#### 4.6 Redshift & Data Warehousing at Scale

- MPP columnar OLAP warehouse. RA3 nodes decouple compute from managed storage (scale independently). Redshift Serverless auto-provisions for variable/ad-hoc workloads.
- Performance: distribution styles (KEY/EVEN/ALL) + sort keys; Concurrency Scaling adds transient clusters for spikes; Spectrum queries S3 without loading; materialized views + result cache.
- Data sharing across clusters/accounts (no copy); zero-ETL integrations (Aurora → Redshift) reduce pipelines; COPY from S3 for bulk load; AQUA/columnar compression.

**TIP** Redshift = complex analytical joins/aggregations at scale (OLAP); Athena = serverless ad-hoc on S3; neither is for OLTP (use RDS/Aurora/DynamoDB).

#### 4.7 Purpose-Built Databases & Migration Tools

- Match engine to workload: relational→RDS/Aurora, key-value→DynamoDB, in-memory→ElastiCache/MemoryDB, document→DocumentDB, wide-column→Keyspaces, graph→Neptune, time-series→Timestream, ledger→QLDB, search→OpenSearch.
- DMS migrates DBs (homogeneous + heterogeneous) with minimal downtime via CDC; SCT (Schema Conversion Tool) converts schema/code for heterogeneous (e.g. Oracle→PostgreSQL, SQL Server→Aurora).
- DMS + SCT + AWS DMS Fleet Advisor / DMS Schema Conversion for large-scale assessment. Combine with DataSync for file data and Snow for offline bulk.

**TIP** Homogeneous lift (SQL Server→RDS SQL Server) → native backup/restore or DMS; heterogeneous (commercial→open-source) → SCT for schema + DMS CDC for data.

## Chapter 5 — Resilience, Disaster Recovery & High Availability

### 5.1 Multi-AZ & Multi-Region Architectures

- Multi-AZ = HA within a region (ASG across AZs + ELB + Multi-AZ RDS/Aurora). Multi-Region = DR, global latency, data sovereignty, and higher availability targets.
- Stateless app tier (externalize state to DynamoDB/ElastiCache/S3) enables horizontal scale + clean regional failover. Global data layer: Aurora Global DB, DynamoDB Global Tables, S3 CRR.
- Active-passive (failover via Route 53 health checks) vs active-active (latency/geo routing + conflict-tolerant data). Active-active raises cost + data-consistency complexity.

**TIP** Cell-based / bulkhead architecture: partition into independent cells to shrink blast radius; shuffle sharding reduces correlated failure impact.

### 5.2 Disaster Recovery Strategies & RTO/RPO

- Four strategies by RTO/RPO/cost: Backup & Restore (hours, cheapest), Pilot Light (core running minimal, scale on failover), Warm Standby (scaled-down full copy, faster), Multi-Site Active/Active (near-zero, priciest).
- RTO = downtime tolerance; RPO = data-loss tolerance. Lower both → higher cost/complexity. Pick per workload tier, not one-size-fits-all.
- Tooling: AWS Backup (centralized, org backup policies, cross-region/cross-account copy), Elastic Disaster Recovery (DRS, block-level replication for fast failover), Aurora Global DB, DynamoDB Global Tables, Route 53 failover.

**TIP** Test DR regularly (game days). Automate failover with Route 53 + health checks + IaC so the recovery region is deployable/reproducible.

### 5.3 Backup, Replication & Data Durability

- AWS Backup centralizes policy-driven backups across EBS/RDS/Aurora/DynamoDB/EFS/FSx/S3/Storage Gateway with cross-region + cross-account copy and Backup Vault Lock (WORM/ransomware protection).
- S3 durability = 11 9s across ≥3 AZs; add versioning + MFA Delete + Object Lock + CRR for protection against deletes/region loss.
- Snapshots (EBS/RDS) are incremental + regional; copy cross-region for DR; Fast Snapshot Restore removes first-touch latency.

**WATCH** Ransomware/insider protection: immutable backups (Vault Lock / Object Lock compliance mode), separate/isolated backup account, and least-privilege on delete actions.

### 5.4 Fault Isolation & Resilience Patterns

- Reduce blast radius: multi-account, multi-AZ, cell-based architecture, shuffle sharding, and static stability (keep running on cached/last-known-good state during control-plane failure).
- Decouple with queues (SQS) + retries with exponential backoff + jitter + circuit breakers to prevent cascading failure and retry storms.
- Graceful degradation: serve reduced functionality (cached/read-only) rather than failing fully. Health checks + auto-recovery + ASG replace unhealthy nodes.

**TIP** Prefer data-plane over control-plane dependencies during failover (control planes fail first at scale) — pre-provision capacity for static stability.

## 5.5 Observability at Scale

- CloudWatch: metrics + alarms + dashboards + Logs Insights; cross-account observability aggregates metrics/logs/traces into a monitoring account. Composite alarms reduce noise.
- CloudTrail = API audit (org trail → central S3, immutable). AWS Config = resource config + compliance + change history. X-Ray = distributed tracing for microservice bottlenecks.
- Centralize: metrics/logs/traces to a dedicated observability account; EventBridge + SSM Automation for auto-remediation; Synthetics canaries for proactive endpoint checks.

**TIP** CloudWatch = what's happening; CloudTrail = who did it; Config = is it compliant; X-Ray = where's the latency.

## 5.6 Chaos Engineering & Fault Injection

- AWS Fault Injection Service (FIS) runs controlled experiments (AZ outage, instance kill, API throttling, latency) to validate resilience and DR assumptions with stop conditions/guardrails.
- Run game days: hypothesize steady state → inject fault → observe → improve. Validates that health checks, failover, and auto scaling actually work under stress.

**TIP** Resilience is proven by testing failure, not by design docs. FIS + game days turn assumed HA/DR into verified HA/DR.

## Chapter 6 — Migration & Modernization

### 6.1 Migration Strategy & the 7 Rs

- The 7 Rs: Rehost (lift-and-shift, e.g. MGN), Replatform (lift-tinker, e.g. RDS), Repurchase (move to SaaS), Refactor/Re-architect (cloud-native), Retire (decommission), Retain (keep on-prem for now), Relocate (VMware Cloud on AWS).
- Rehost is fastest/lowest-risk (modernize later); refactor is highest value/effort. Sequence: quick wins first, modernize iteratively (strangler-fig).
- AWS Migration Acceleration Program (MAP): Assess → Mobilize → Migrate & Modernize, with funding + tooling + Migration Hub tracking.

**TIP** Choose R per app by business value + technical debt + urgency; don't refactor everything up front — rehost to get out of the data center, then modernize.

### 6.2 Discovery & Planning

- Application Discovery Service (agent-based or agentless) collects utilization/dependencies; Migration Hub tracks portfolio + progress across tools; Migration Evaluator builds the business case/TCO.
- Map dependencies to migrate in waves (move tightly-coupled apps together). Assess with Fleet Advisor (DB inventory) and right-size targets before cutover.

**TIP** Dependency mapping avoids breaking chatty inter-app calls — migrate coupled components in the same wave to prevent cross-DC latency.

### 6.3 Server & Database Migration

- AWS Application Migration Service (MGN) = primary rehost tool: continuous block-level replication → test → cutover with minimal downtime (replaces SMS/CloudEndure).
- DMS migrates databases with CDC for near-zero downtime; SCT converts schema/code for heterogeneous migrations (Oracle/SQL Server → Aurora/PostgreSQL).
- Homogeneous DB (same engine): native backup/restore or DMS. Heterogeneous: SCT (schema) + DMS (data + ongoing CDC), validate with DMS data validation.

**TIP** MGN for servers, DMS for databases, SCT when the engine changes. Keep source live during CDC, cut over after validation, keep a rollback window.

### 6.4 Data Transfer (DataSync, Transfer Family, Snow)

- DataSync = fast, scheduled/one-time bulk transfer (NFS/SMB/HDFS/S3/EFS/FSx) with encryption, validation, and bandwidth throttling — for online migration/replication.
- AWS Transfer Family = managed SFTP/FTPS/FTP/AS2 front door to S3/EFS (for partner/B2B file exchange without running servers).
- Snow Family for offline bulk when network transfer is too slow (Snowcone/Snowball Edge). Rule of thumb: if online transfer would take > ~1 week, evaluate Snow.

**TIP** Bandwidth math drives the choice: 100 TB over 1 Gbps  $\approx$  12+ days  $\rightarrow$  Snowball. Ongoing sync/replication  $\rightarrow$  DataSync; partner file exchange  $\rightarrow$  Transfer Family.

## 6.5 Modernization Patterns

- Strangler-fig: incrementally replace a monolith by routing slices of traffic to new services (behind API Gateway/ALB) until the monolith is retired.
- Modernize to containers (ECS/EKS) for portability + density, or serverless (Lambda/Step Functions/EventBridge) for event-driven + no-ops scaling.
- Database modernization: commercial  $\rightarrow$  open-source (Aurora/PostgreSQL) via SCT+DMS to cut licensing; break shared DBs into per-service data stores.

**TIP** Refactor for measurable outcomes (cost, agility, scale) — not for its own sake. Wrap-and-replace beats big-bang rewrites for risk.

## 6.6 Hybrid & VMware Cloud

- VMware Cloud on AWS (and relocate strategy) = run VMware SDDC on AWS bare metal for fast lift with no re-platforming; connect to native AWS services.
- Outposts extend AWS into your data center (low latency, local data residency); Local Zones/Wavelength for edge/5G. Storage Gateway + DataSync bridge hybrid data.

**TIP** Relocate (VMware Cloud) when you need to exit a data center fast without changing the VM stack; modernize to native services afterward.



## Chapter 7 — Cost Optimization, Deployment Automation & Exam Prep

### 7.1 Cost Optimization at Scale

- Commitment discounts: Compute Savings Plans (most flexible — across regions/families/Fargate/Lambda), EC2 Instance SP, Reserved Instances (up to 72%). Spot (up to 90%) for fault-tolerant.
- Right-size (Compute Optimizer), eliminate waste (idle/unattached EBS/EIP, old snapshots), schedule non-prod off-hours, and use serverless/auto-scaling to match demand.
- Storage tiering (S3 Intelligent-Tiering + lifecycle, gp3 over gp2), data-transfer reduction (VPC endpoints, CloudFront, same-AZ), and Graviton for price/performance.

**TIP** Layered strategy: Savings Plans for baseline + Spot for elastic + On-Demand for spiky remainder; tag everything for allocation.

### 7.2 Cost Governance & Visibility

- Consolidated billing (org) pools volume discounts + shares Savings Plans/RIs. Cost allocation tags + Cost Categories group spend by team/project/environment.
- Cost Explorer (analyze/forecast + SP/RI recommendations), Budgets (threshold alerts + actions), Cost & Usage Report (granular, query via Athena/QuickSight), Cost Anomaly Detection (ML spike alerts).
- Governance: SCPs restrict expensive services/regions; Budgets Actions can auto-apply restrictive policies when thresholds are breached.

**TIP** Establish FinOps: tagging standards (enforced via Tag policies) + per-account budgets + anomaly detection + regular Trusted Advisor reviews.

### 7.3 CI/CD & Deployment Automation

- AWS CodePipeline (orchestration) + CodeBuild (build/test) + CodeDeploy (deploy) + CodeArtifact (packages); or GitHub Actions/GitLab. Multi-account pipelines deploy via cross-account roles.
- CodeDeploy supports in-place + blue/green for EC2/ECS/Lambda with automatic rollback on CloudWatch alarms. CloudFormation StackSets deploy IaC across accounts/regions.

**TIP** Separate build (tooling account) from deploy targets (workload accounts) with cross-account roles; gate promotions with approvals + automated tests.

### 7.4 Deployment Strategies

- All-at-once (fast, downtime/risk), Rolling / rolling-with-batch (gradual), Immutable (new instances, safe rollback), Blue/Green (parallel env, instant switch + rollback), Canary (small % first).
- Traffic shifting: Route 53 weighted, ALB weighted target groups, Lambda alias weights, CodeDeploy canary/linear configs. Feature flags decouple deploy from release.

**TIP** Blue/Green = instant rollback via traffic swap (needs 2x capacity briefly); Canary = catch issues on a small cohort before full rollout.

## 7.5 Infrastructure as Code

- CloudFormation (declarative templates, StackSets for multi-account/region, drift detection, change sets, Hooks for proactive guardrails). CDK generates CloudFormation from code (TS/Python/etc.).
- Reusability: nested stacks, modules, Service Catalog (govern approved products for self-service), and parameter/config separation per environment.

**TIP** IaC everywhere → reproducible DR regions, consistent multi-account baselines, and reviewable/auditable changes. StackSets + Control Tower for org-wide rollout.

## 7.6 SAP-C02 Exam Overview & Domains

- 75 questions, 180 minutes, scaled score 100–1000, pass = 750. \$300 USD. Multiple choice + multiple response. Valid 3 years. Recommend 2+ yrs hands-on AWS.
- Domains: (1) Design Solutions for Organizational Complexity 26%, (2) Design for New Solutions 29%, (3) Continuous Improvement for Existing Solutions 25%, (4) Accelerate Workload Migration & Modernization 20%.
- Questions are long multi-requirement scenarios — often several valid answers where only one is BEST for the stated qualifier (cost, least operational overhead, fastest RTO, most secure).

**TIP** Parse the constraints first (RTO/RPO, cost, ops overhead, compliance, timeline), then eliminate options that violate any hard constraint before optimizing the soft one.

## 7.7 Exam Strategy & Career Path

- Time budget ~2.4 min/question — flag long ones and return. Read the final question sentence first to know the actual ask; watch qualifiers (MOST cost-effective / LEAST overhead / fastest).
- High-yield areas: Organizations/SCPs, Transit Gateway/DX/hybrid DNS, cross-account IAM/federation, migration tooling (MGN/DMS/SCT/DataSync), DR strategies, multi-region data, cost governance.
- Whitepapers/FAQs: Well-Architected + all six pillars' deep dives, DR, Organizations, Hybrid Networking, Migration (7 Rs). Do timed full-length practice; target 80%+ before booking.

**TIP** SAP-C02 is scenario judgment, not trivia — build a multi-account network (TGW+DX+SCPs) and a migration (MGN/DMS) in a lab to internalize the trade-offs.